

Unit Testing with JUnit 5 and Mockito - Interview Sheet

Definitions

Unit Testing: Testing individual units/components of a software in isolation.

JUnit 5: A popular testing framework for Java, supporting annotations and assertions.

Mockito: A mocking framework for unit tests in Java, used to simulate behavior of dependencies.

Common Annotations and Their Usage

@Test: Marks a method as a test case.

@BeforeEach: Runs before each test method.

@AfterEach: Runs after each test method.

@ExtendWith(MockitoExtension.class): Enables Mockito support in JUnit 5.

@Mock: Creates a mock instance of a dependency.

@InjectMocks: Injects mock dependencies into the class under test.

@SpringBootTest: Loads the full Spring application context (used for integration tests).

@WebMvcTest: Loads only web layer components (used for controller testing).

@DataJpaTest: Loads only JPA components (used for repository testing).

@MockBean: Creates a mock bean in the Spring context.

@Autowired: Injects Spring-managed beans.

Common Interview Questions

1. What is the difference between unit testing and integration testing?
2. How does Mockito help in unit testing?
3. What is the purpose of @Mock and @InjectMocks annotations?
4. How do you test a service class that depends on a repository?
5. What is the difference between @Mock and @MockBean?
6. How do you verify method calls in Mockito?
7. What is the role of @ExtendWith(MockitoExtension.class)?

8. How do you handle exceptions in unit tests?
9. What are the advantages of using JUnit 5 over JUnit 4?
10. How do you test private methods in a class?